

Week 3: Program Flow Control

Before You Start

Ensure you have completed all tasks from Week 2. Any issues seek help from the tutors and lecturers.

Hint: It is good practice to plan your programs first on paper using pseudocode. When we say pseudocode, we mean code that is halfway between English and a programming language, such as Python.

Read more about it here: <https://en.wikipedia.org/wiki/Pseudocode>

Attempt to complete this set of tasks before your next tutorial.

Introduction

This session aims to familiarise you with flow control, lists and iteration.

Question 1: Comparing Numbers

Write a program to compare two numbers (integers).

1. On paper, devise a routine using pseudocode to ask the user for two numbers, compare them and then print a message indicating which one is bigger than the other (or if they are both the same) with an appropriate message.
2. Carefully work out the structure of your conditional (**if**) statement before you begin implementation of your procedure.
(you could use a truth table to help, if you want)
3. Now implement your algorithm in Python!

Question 2: Eligible Voters

Write a program to check if someone is eligible to vote in the UK.

1. Create a program to ask the user for their name, age and country of citizenship. Your program should then process this information to determine if the named person can vote according to the following rules:
 - a. If the person is under 18, they are not allowed to vote. Print out a message which includes their name and states that they are not yet old enough to vote.
 - b. If the person is over 18 and their country of citizenship is the UK or Britain, print out a message including their name that states they are entitled to vote.
 - c. Otherwise, if they are over 18 but their country of citizenship is not the UK or Britain, print out a message stating that they are not entitled to vote.
2. Test your program with suitable test cases.
(a truth table could help to test your results)

Question 3: Fizz Buzz

Create a program that asks the user for a number and then outputs a message based on the properties of that number:

1. If the number is divisible by 3, print the message `Fizz`.
2. If the number is divisible by 5, print the message `Buzz`.
3. If the number is divisible by both 3 and 5, print the message `Fizz Buzz`.
4. If the number is not divisible by either 3 or 5, print the number itself.

Hint: Think about how you will test if a number is divisible by another number. Do some research on the modulo operator (%) and think carefully about how it can help us here. Plan your conditional statements out on paper before you get started coding.

Question 4: Sorting

You're tasked with writing a program that sorts 3 numbers.

1. Create a program to ask the user for three numbers (called `first_num`, `second_num`, `third_num`) and print them out in order from smallest to largest.
2. Test your code using the following test cases. You must enter the numbers in the exact order given for each test case. So, for the first test case (*test case a*); `first_num = 87`, `second_num = 99`, and `third_num = 11`. Run your program for each test case to ensure it works correctly.
 - a. 87, 99, 11
 - b. 87, 11, 99
 - c. 99, 87, 11
 - d. 99, 11, 87
 - e. 11, 99, 87
 - f. 11, 87, 99

Question 5: Grades

Write a program that assigns grades to students based on their marks:

1. Create a program that asks the user for a mark from 0 to 100 and prints a corresponding grade using the table to the left.
2. Devise a test case scheme and test that your program produces the correct output!
3. This might be trickier than you think. What if the user enters a negative number? What if they enter a number greater than 100? What if the thing they enter isn't a number at all? Make sure your program can deal with all of these cases.

Mark	Grade
0	N/S
1-39	F
40-49	D
50-59	C
60-69	B
70-79	A
80-100	A*

Question 6: Who's that Pokémon?

Write a program that asks the user for the type of a Pokémon and the first letter of its name, then makes suggestions about what that Pokémon might be called.

1. Ask the user for the type of the Pokémon.

This can be one of:

- a. Fire
- b. Water
- c. Grass
- d. Electric

2. Ask the user for the first letter of that Pokémon's name.
3. Based on the information you've got, output suggestions based on the table to the right.

Type	First letter	Suggestion
Water	S	Squirtle
Water	T	Tentacool
Fire	C	Charmander
Fire	M	Moltres
Grass	B	Bulbasaur
Grass	O	Oddish
Electric	P	Pikachu
Electric	V	Voltorb

Question 7: Comparing Many Numbers

Improve your answer to question 1 by allowing multiple comparisons per program run.

1. Copy your answer to question 1 and save it in a different file.
2. Adjust your answer to use a loop. Keep asking for pairs of numbers and comparing them until the user types "quit" instead of a number.
3. Pay particular attention to your casts here. Can you wrap the casts around your `input()` calls and still have the program work as expected? If not, where should you put them instead?

Question 8: Sum of Numbers at Even Indices

Write a program that sums up only the numbers in a list that are at even indices.

1. Declare a list containing the values 14, 5, 19, 20, 21, 66 and 89 in this exact order.
2. Using a loop, add up only the values at even indices in this list. So that's:
 - a. 14 (index 0)
 - b. 19 (index 2)
 - c. 21 (index 4)
 - d. 89 (index 6)